



Identity Based Encryption Scheme



Use Case Scenario

- Imagine a university professor who wants to send confidential grades to a student.
- Instead of searching for the student's digital security certificate, the professor can simply use the student's **ID number** or **university email** as the public key to encrypt the file.
- Only the student, upon proving their identity to a central authority, will be able to get the private key needed to read those grades



Identity-Based Encryption (IBE)

- In a traditional encryption setup, if you want to send a secure message to someone, you must first obtain their **Public Key** from a directory or a certificate authority. This key is usually a long, confusing string of data that is difficult to remember or verify manually.
- **What is IBE?** A system where you can use **any text** (like an email address) as a public key to encrypt a message.
- **Working:**
 1. Encrypt using receiver's (Alice) identity – e.g., `alice@example.com`
 2. Private key from authority – Alice proves her identity and gets a private key
 3. Only Alice decrypts – She uses her private key to read the message
- **Math Behind It (Simple):**

Uses **Bilinear Pairing** – a special map that links two values so only the matching private key can unlock the encryption
- **Why It Matters:**
 - With IBE, you don't need to ask the receiver for their key beforehand. No need for certificates
 - You can encrypt a message for someone even if they have never set up an encryption account or generated a key pair
- **Who Made It Possible?**

Boneh & Franklin (2021) – first working IBE system

Bilinear Pairing

What is it?

A special math function (like a magic bridge) that connects two different math worlds:

- **World (G_1)** → Contains points on an elliptic curve, where the primary action is adding points together.
- **World (G_2)** → Consists of numbers, where the primary action is multiplying them.

Key Property:

1. If you perform an action multiple times in World 1 and then "cross the bridge," you get the same result as crossing the bridge first and then doing the math in World 2.

The Simple Example:

Path A (Work Before Pairing): You have 10 Gold Coins. You double them in World 1 first (so you have 20 Gold), and then walk across the bridge to exchange them for Silver.

Path B (Pairing First): You take your 10 Gold Coins and walk across the bridge first to get your Silver. Once you are in World 2, you double that Silver amount using World 2's math. **You end up with the exact same amount of Silver in both scenarios.**

Why is it useful?

- Helps build **Identity-Based Encryption (IBE)**: allows you to use an email address as a public key
- Enables **short digital signatures**: creates very small, data-efficient digital "stamps" to prove identity.
- Used in **secure key exchange**: helps two parties agree on a secret code over the internet



Identity Based Encryption scheme

Setup (Done Once by Private Key Generator-PKG)

- PKG = Trusted authority (like a post office headquarters).

What they do:

- Announce mathematical "laws" of the system to everyone; so that they can communicate.
- Keep one master secret key safe (like a master stamp - never shared with anyone).

Think: Post office announces services, keeps master stamp hidden.

Extract (Done for Each New User)

User gives their identity (e.g., email: alice@example.com).

PKG uses:

- Master secret key and User's identity
 - Output: User's secret key (given only to the user).

You get your own private key based on your identity.



Identity Based Encryption scheme

Encrypt (Done by Sender – Bob)

- Bob wants to send a secret message to Alice

Bob needs:

- His message (plaintext)
- Alice's identity (alice@email.com)
- Public rules (from PKG)
- A random secret value (chosen by Bob) to ensure the ciphertext is unique every time.

Bob does:

- Puts everything into the **Encrypt machine** (magic box)
- Gets **encrypted message** (locked box)
- Sends it to Alice

Decrypt (Done by Receiver – Alice)

- Alice receives the encrypted message

Alice needs:

- The encrypted message (locked box)
- Public rule
- Her own **secret key** (PKG gave her earlier)

Alice does:

- Puts everything into the **Decrypt machine**
- Gets back the **original message**



Construction of the Scheme

Step 1: Choose Mathematical Tools

- **Bilinear pairing e :** A special math rule that connects two number groups.: Taking 2 inputs from first group $G_1 \times G_1 \rightarrow G_2$
- **Random generator point P from G_1 (anchor point):** a starting value in G_1 that everyone uses as a common reference for their calculations

Step 2: Create Master Key (by PKG)

- Pick random secret s (master secret key – only PKG knows)
- Compute $P_{pub} = P^s$ (public key from master secret)

Step 3: Add Hash Functions (The translators)

Hash	What it does	Input → Output
H	Converts numbers to messages	12345... → "Hello"
G	Converts identity to number	alice@email.com → 6789



Construction of the Scheme

Step 4: Public vs Secret

Public Parameters (Everyone knows including Bob)

p, n – system parameters defining system boundaries

P – generator point

P_{pub} – public key

G, H – hash functions

Master Secret (Only PKG knows)

s – the secret number

Message space: $M = \{01\}^n$, (the message can be any sequence of bits (0s and 1s) of a certain length n)



Construction of the Scheme

How a New User Registers (e.g., `alice@email.com`)

Step 1: Identity → A unique dot on a Curve

- System takes the email address `alice@email.com`
- Runs it through a special formula (hash function) to turn it into a **single point** on an elliptic curve
- This point is called **Q_ID** (the “identity dot”)

Like converting a name into a unique dot on a special graph

Step 2: Master Secret + Identity Dot → Private Key

- The PKG (Key Generator) holds a **master secret key** s (kept hidden)
- The PKG takes Alice's point (Q_ID) and adds it to itself s times.
- This calculation creates a new point **d_ID** – the Alice's **private key**

Like stamping the identity dot with the PKG's secret master stamp

Construction of the Scheme

Encrypt: Send Message M to Identity ID

Step 1: Identity $\rightarrow$ Same Unique Dot

- Bob takes the recipient's identity (e.g., alice@email.com)
- Using a special formula, he turns that email into the exact same dot (**Q_ID**) on a mathematical curve that was used when Alice's private key was first created.

Step 2: Compute Shared Secret Base

- Bob needs to build the foundation of the lock. He uses the "magic pairing machine" (bilinear pairing e) to combine two public items:
 - **Q_ID** (Alice's identity dot)
 - **P_pub** (PKG's public key, known to everyone)
- Formula: Shared Secret Base **$g_ID = e(Q_ID, P_pub)$**

This creates a special lock that only Alice can open — made from her name and the PKG's public stamp

Now use this lock (g_ID) to encrypt the message M so only Alice can decrypt it.



Construction of the Scheme

Step 3: Add a Random Secret Ingredient

- Pick a random number r (known only to Bob)
- This makes **every encryption different**, even for the same person

Like adding a pinch of random salt same recipe, unique result every time

Step 4: Create Ciphertext (2 Parts) Bob now packages everything into the final Ciphertext C , which is the actual "locked box" sent over the internet. The formula is:

$$C = \langle P^r, M \oplus H(g_{ID}^r) \rangle$$

Part	What it is	Purpose
First	P^r	Public clue for decryption
Second	$M \oplus H(g_{ID}^r)$	Actual Message hidden with secret mathematical mask

- P is public anchor point
- H is the hash function that turns this into a message-sized string
- $\oplus$ is XOR – we **mask** the message with this hash

Construction of the Scheme

Decrypt: Open Ciphertext $C = \langle U, V \rangle$ Using Private Key d_{ID}

What Alice Has:

- Ciphertext $C = \langle U, V \rangle$ **from Bob**
- Her private key $d_{ID} = Q_{ID}^S$ (from PKG)
- Public parameters known to sender and receiver (pairing e , hash H)

Decryption Steps

Step 1: Recover Shared Secret

- Alice uses her private key and the first part of the ciphertext U :
Secret = $e(d_{ID}, U)$

This gives exactly the same g_{ID}^r that Bob used to hide the message!

Like using your private key to unlock the same "secret ingredient" the sender added

Construction of the Scheme

Step 2: Unmask the Message

- She takes the second part V and XORs it with the hash of that secret:

$$M = V \oplus H(\text{Secret})$$

The mask cancels out the original message appears!

Implementation

- The implementation can be possible using PBC library (if you are conformable with C) or you can go for JPBC (if you are comfortable with Java)